

Autopilot Tuning Report

How the gains were found: measure first, change one thing, prove it in simulation, repeat.

The Method — no guesswork

- 1 Build a truthful physics model.** A JSBSim flight-dynamics model of the Talon 250G was created from the real specs (250 g, 620 mm span, 6.6 dm², 5×5 pusher) and validated against known behaviour: stall 6.7 m/s, top speed 16 m/s, glide ratio ~9.
- 2 Derive first-guess gains from physics.** Cruise throttle, climb feedforwards and limits were *calculated* from the model (e.g. measured 28 % throttle at 12 m/s), not copied from another plane.
- 3 Fly repeatable test patterns headlessly.** Scripted flights — hand launch, +30 m altitude step, laps of *Oval 1-2* — measured altitude error, airspeed and bank continuously, scored as capture time, dip, balloon and overshoot, split into "straights" vs "turns".
- 4 Change one gain, re-fly, compare numbers.** Only changes that measurably improved the numbers (without breaking another test) were kept. Where no gain could fix the physics, the *firmware itself* was improved: a bank-angle energy feedforward and a constant-deceleration altitude capture were added to `energy_ctrl.c` (both default-off, upstream-safe).

What was changed, and why (plain language)

PARAMETER	OLD → FINAL	WHY (IN PLAIN TERMS)
POWER_CTL_BAT_NOMINAL	11.7 → 7.2 V	Was a 3-cell value on a 2-cell plane. The controller scales throttle with battery sag — wrong voltage = wrong scaling.
AUTO_THROTTLE_NOMINAL_CRUISE_THROTTLE	0.60 → 0.28	The measured throttle that actually holds 12 m/s level. Starting from the true operating point means the adaptive loops barely have to correct anything.
AUTO_THROTTLE_NOMINAL_CRUISE_PITCH	0 → 0.06 rad	The nose sits ~3.5° up in cruise on this wing; telling the controller up front avoids a sag after every mode change.
AUTO_THROTTLE_CLIMB_THROTTLE_INCREMENT	0.30 → 0.17	"Extra throttle per m/s of climb" — computed: (100 % – 28 %) over the 4.3 m/s max climb. Correct feedforward keeps feedback small.
AUTO_THROTTLE_PITCH_OF_VZ_PGAIN	0.01 → 0.085	"How much nose-up per m/s of climb" — simply geometry: climb angle = climb rate ÷ airspeed = 1/11.5.
ALTITUDE_PGAIN	0.05 → 0.60	Altitude stiffness. ETECS has no altitude integrator , so standing errors (turn sag) shrink as 1/PGAIN. With the capture law below taming approaches, 12× stiffer was possible without harshness — tighter turns <i>and</i> crisp final metres.
ALTITUDE_ACCEL_CAPTURE (new)	off → 0.8 m/s ²	New firmware feature: level off at a <i>constant, gentle deceleration</i> instead of the exponential crawl. Level-off after +30 m: 17.8 s → 4.0 s (–78 %) , +0.05 m overshoot, same airspeed.
ALTITUDE_MAX_CLIMB	1.0 → 2.0 m/s	Ceiling on commanded climb: brisk, yet always leaves throttle for the speed loop; keeps altitude changes gradual.
ALTITUDE_MAX_CLIMB_DOT (new)	2 (hardcoded) → 4.0 m/s ²	Was a magic number in the code. Un-hardcoded; 4 lets the altitude loop <i>catch</i> a turn-entry dip and <i>unwind</i> the correction before it balloons past target.
MAX_ACCELERATION	0.8 → 0.35 g	Never ask for more than the motor can deliver (physics: 0.44 g max) — prevents saturation and wind-up.
ENERGY_TOT_P / I	0.45 / 0.20	Throttle response to "total energy is wrong". Raised so the motor answers the extra drag of a banked turn quickly.
ENERGY_DIFF_P / I	0.60 / 0.10	Pitch response to "energy is in the wrong place". In turns the energy hid in airspeed while height sagged — this trades it back promptly.
ENERGY_BANK_THROTTLE_GAIN (new)	off → 0.36	New firmware feature: pays the <i>predictable</i> drag cost of a banked turn the moment the wing rolls, not after height is lost. Sized from the drag polar; trimmed 0.40→0.36 once the stiffer PGAIN carried its share.
ENERGY_BANK_WASHOUT_GAIN / TAU (new)	off → 0.85 / 2.0 s	Transient half of the bank feedforward: strong push <i>only while rolling in</i> (bridges motor lag), zero mid-turn, <i>negative</i> on roll-out — which killed the exit "balloon" above target height.
ENERGY_BANK_PITCH_GAIN (new) + PITCH_OF_ROLL	off → 0.19 / 0 → 0.04	Banked wings need more lift (load factor 1/cosφ). New gain raises the nose from the <i>commanded</i> bank (leads the turn); the small classic term follows the <i>measured</i> bank. Split so neither over-pitches alone.
PITCH_IGAIN / ROLL_IGAIN	6→400 / 20→100	The attitude "trim knobs". Old time constants were 1000 s / 550 s — effectively off. Now steady offsets (CG, asymmetry) trim out within seconds.
ROLL_KFFA / PITCH_KFFA / REF_W_Q	10→100 / 5→20 / 5→7	Feedforward "muscle memory" plus a faster pitch reference model: needed deflection is applied immediately, and pitch demands arrive while the roll is still happening.
THROTTLE_SLEW_LIMITER	0.8 → 0.25 s	A 250 g pusher can use its throttle quickly and safely; the old limit smeared every correction over nearly a second.
SDP3X_LOWPASS_TAU	0.09 → 0.15 s	Airspeed filtering was <i>below</i> the driver default; the speed loop is far slower than the 1 Hz cut-off — stronger smoothing is free noise rejection.

Final measured outcome (Oval 1-2, 30 m radius, 33° banks): worst height dip in turns **–0.83 m → –0.17 m**, worst balloon **+0.19 m**, turn mean **–0.02 m**, straight legs **±0.15 m**; airspeed pinned at **12.0 ± 0.4 m/s** throughout. +30 m step: level-off **17.8 s → 4.0 s**, overshoot **+0.05 m**, no oscillation. Hand launch climbs away hands-off.

Why Energy Control — and What to Watch

Why (E)TECS suits a small UAV

- **One budget, two currencies.** TECS treats height and speed as one energy account: $E = mgh + \frac{1}{2}mv^2$. Throttle adds/removes energy; pitch only decides whether it is stored as *height* or *speed*. That matches how an airplane actually works.
- **Light airframes trade energy fast.** A 250 g foamie converts speed↔height in a heartbeat: a classic "pitch for altitude, throttle for altitude too" controller constantly fights itself, TECS never does — each actuator has exactly one job.
- **Stall protection by construction.** Speed is a first-class citizen: the controller floors its setpoint at $1.23 \times \text{stall}$, and when the throttle saturates it deliberately gives up climb rather than airspeed. On a plane that stalls at 6.7 m/s that discipline is survival.
- **It learns the airframe ("adaptive").** The nominal cruise throttle and pitch are slowly re-estimated in flight. As the 2S Li-Ion sags from 8.4 V to 6.6 V — or a different battery shifts the CG — the trim follows automatically, so the plane flies the same at the end of the pack as at the start.
- **Wind-gust economy.** A gust converts to a speed error, absorbed mostly by re-distributing energy (pitch) rather than motor bursts — smoother flight, longer endurance.
- **Feedforward-friendly.** Because the energy bookkeeping is explicit, *predictable* costs (banked turns, climbs) can be paid in advance with physically-sized feedforwards — the feedback loops then only handle genuine surprises like gusts.

Rule of thumb worth keeping: feedback fixes what you did not foresee; feedforward pays for what you can compute. Every improvement in this campaign that actually moved the numbers was a feedforward sized from the airframe's physics — the feedback gains merely needed to be *awake* (integrators) and *calm* (no saturation).

Risks & points of attention

- **Airspeed sensor is mission-critical.** ETECS is blind without it — a blocked or icy pitot means the controller chases a phantom speed (we reproduced this in sim: a missing airspeed feed caused a full-throttle dive). Pre-flight: blow-check the SDP33, watch the reading during the launch sprint.
- **On-ground adaptation poisoning.** After landing, airspeed reads ~ 0 while the integrators are still allowed to "learn" — the cruise throttle can wind up (known airframe TODO: aircraft may throttle up after touchdown). Kill the throttle promptly at touchdown.
- **No altitude integrator.** The altitude loop is P-only: any sustained disturbance leaves a standing error $\sim 1/\text{PGAIN}$ in size. The high PGAIN 0.6 used here is only comfortable *because* the capture law removes its harshness on approaches — do not copy the value without the capture feature.
- **Capture law constraints.** Keep `ALTITUDE_ACCEL_CAPTURE ≤ ALTITUDE_MAX_CLIMB_DOT` (or the rate limiter lags the level-off and causes overshoot) and note the sqrt branch only engages when `PGAIN > CAPTURE/MAX_CLIMB` — with a low PGAIN the feature silently does nothing.
- **Do not compensate twice.** The bank pitch feedforward and the classic `H_CTL_PITCH_OF_ROLL` both raise the nose in turns. They are deliberately split here (0.19 leading / 0.04 trailing); enabling both at full size will balloon every turn.
- **Feedforwards must match reality.** All were computed from the model; if the prop, weight or battery changes materially, recompute them — wrong feedforward makes every manoeuvre start with a lurch.
- **Respect saturation.** `MAX_ACCELERATION` and `ALTITUDE_MAX_CLIMB` are deliberately set *inside* the physical envelope — "asking for more" only winds up integrators.
- **Sim-to-real gap.** These gains were proven against a validated model, but a real airframe adds vibration, sensor noise and servo slop. Start the first flights with the bank feedforwards at $\sim 70\%$ of the sim values and walk them up via `dl_settings`.

Two New Features — and How to Tune the Next Airframe

1 • Bank-angle energy feedforward

In a coordinated turn the wing must produce lift for load factor $n = 1/\cos\phi$ and induced drag grows with n^2 — a perfectly predictable energy cost. The feedforward pays it as *the wing rolls* (using the commanded bank, which leads the actual turn), in two parts:

```
/* u = 1/cos^2(phi) - 1, bank capped at 60 deg */
throttle += K_steady * u          /* exact turn cost */
          + K_washout * highpass(u) /* roll-in punch,
          zero mid-turn, NEGATIVE on roll-out (kills
          the exit balloon). tau = 2 s */
pitch    += K_pitch * (1/cos(phi) - 1);
```

- **Talon 250G values:** steady 0.36, washout 0.85, τ 2.0 s, pitch 0.19.
- **Safety:** bounded (60° cap), purely additive, never integrated (cannot pollute the adaptation), vanishes wings-level. Defaults to off.

2 • Constant-deceleration altitude capture

The classic proportional law levels off exponentially — the last metres crawl (17.8 s here). The new law approaches at full climb, then bleeds climb rate at a constant, chosen deceleration a , blending tangentially into the linear law for the final centimetres:

```
/* outside |err| <= a/P^2 : */
climb_sp = sign(e) * sqrt(2a*(|e| - a/(2P^2)))
/* inside: classic climb_sp = P * e
   (continuous in value AND slope) */
```

- **Result:** level-off 4.0 s (−78 %), +0.05 m overshoot, airspeed untouched — the altitude-vs-time curve becomes a clean parabola instead of a long tail.
- **Both features** live in `energy_ctrl.c`, documented in `guidance_energy.xml`, tunable in flight via `dl_settings`, and default off — candidates for an upstream pull request after a flight-test campaign.

Recipe: tuning another airframe

- 1 **Measure the plane first.** Weigh it. Find true stall speed, best-glide speed and ratio, top speed, max climb rate, and — crucially — the *throttle fraction that holds cruise speed level*. Sim or careful manual flight both work.
- 2 **Fill in the physics, not defaults:** battery voltage under load; `NOMINAL_CRUISE_THROTTLE` = measured; `CLIMB_THROTTLE_INCREMENT` = $(1 - \text{cruise})/\text{max-climb}$; `PITCH_OF_VZ` $\approx 1/\text{airspeed}$; `MAX_ACCELERATION` below $(T-D)/W$; airspeed floor at $1.23 \times \text{stall}$.
- 3 **Wake up the attitude loops.** Verify the pitch/roll integrators actually act within seconds (compute `SUM_ERR_MAX/IGAIN` — if it is minutes, they are decorative), give the feedforwards (KFFA) real values, then fly straight and level hands-off before touching anything else.
- 4 **Tune energy loops on a step.** Command ± 30 m: raise `ENERGY_TOT` P/I until throttle answers promptly without hunting, then `ENERGY_DIFF` P/I until speed stays pinned during the climb. Watch the airspeed trace, not the altitude trace.
- 5 **Then turns.** Fly ovals; log altitude split by bank angle. Size the steady bank-throttle gain from the dip, add washout for the entry transient, and only then trim pitch-of-bank. One knob per flight.
- 6 **Last: capture crispness.** Enable `ALTITUDE_ACCEL_CAPTURE` at ~ 0.5 , raise `ALTITUDE_PGAIN` until the terminal region feels tight (remember: the sqrt branch needs `PGAIN > a/\text{MAX\_CLIMB}`), and re-run the turn test — a stiffer `PGAIN` usually means the steady bank gain can come down.
- 7 **Regress everything, every time.** A change that helps captures can break turns (it did here, twice). Keep the scripted tests; numbers beat impressions.

The one unbreakable rule: change one parameter at a time, and keep a written (or scripted) record of the number it moved. Two knobs at once means neither is understood — and an autopilot that "mostly works" for reasons nobody knows is a future crash.